

PageIndex Implementation Flow

```
document → page text → ToC detection → section-list construction → verification → tree  
conversion → retrieval-ready document record
```

1. **Parse pages:** Extract plain text from the PDF.
2. **Detect ToC pages:** Decide whether the document already contains usable ToC pages.
3. **Choose section-list building path:**
 - **ToC with page numbers:** Convert the detected ToC into JSON, then map printed page numbers to physical PDF pages.
 - **No usable ToC:** If no ToC is found, or the detected ToC has no usable page numbers, infer the hierarchy directly from page-marked text.
 - **Fallback ToC-title mapping:** If the ToC-with-page-numbers path fails verification, reuse the detected ToC titles and ask the LLM to locate their physical start pages from page-marked text.
4. **Group pages:** Long documents are split into token-safe groups before prompting. This keeps each LLM call within context limits and avoids forcing the model to read the whole document at once.
5. **Verify and repair indexes:** Generated section boundaries are checked against the document text. If a section appears to start on the wrong page, the system searches nearby pages and asks the LLM to repair the boundary.
6. **Convert section list to tree:** The verified section list is converted into a recursive hierarchy. Each node receives a **start_index**, an **end_index**, and child nodes when the section contains smaller sections.
7. **Split large nodes:** Sections that are too large are processed again at a finer level, creating deeper subtrees only where the document needs more detail.
8. **Add retrieval fields:** The final tree can include **node_id**, raw **text**, node **summary**, and a document-level description depending on the indexing settings.
9. **Store retrieval-ready data:** The system stores the document record, tree, page count, extracted page text, and optional descriptions so later retrieval can return either the compact tree or the raw evidence.
10. **Support query-time retrieval:** At query time, the system can expose document meta-data, a tree without raw text, and selected raw page evidence for answer generation.

LLM usage note: These steps can use the same model, but they are separate LLM calls. The pipeline passes explicit data between steps, such as extracted page text, ToC JSON, selected **doc_id**, selected nodes, and retrieved evidence. Some long-generation operations pass an explicit short continuation history inside that operation, but the overall pipeline does not rely on one shared hidden chat history.

Indexing Step 1: Create PageIndex Tree for Each Document Using LLM

- **Input:** Document
- **Output:** JSON-based hierarchical PageIndex tree
- **LLM role:** The LLM is used to detect ToC pages, transform ToC text into JSON, infer section boundaries, locate section starts, repair incorrect page indexes, and generate optional summaries/descriptions.

Implementation detail:

1. **Extract page text:** Convert the PDF into an ordered list of page text and token counts.

- The actual extraction output is a list of (`page_text`, `token_count`) pairs.
- The physical page index is implied by the list position: the first pair is physical page 1, the second pair is physical page 2, etc.

```
[
  ("Annual Report 2023 ...", 742),
  ("Financial Stability ...", 691)
]
```

2. **Detect ToC pages:**

- (a) Ask the LLM to label each checked page as `toc_detected`: "yes" or "no".

Input: The extracted text of one checked page. The token count is kept for later grouping, but is not sent to this ToC-detection prompt.

```
Annual Report 2023 ...
```

LLM Output: Detected ToC pages are stored as zero-based list indexes.

```
toc_page_list = [0, 1] // physical PDF pages 1 and 2
```

Remark:

- Abstract, summary, notation list, figure list, and table list are not treated as ToC.
- If no ToC is found in the first `toc_check_page_num` pages, 20 by default, the result is an empty ToC page list.
- Page-check behavior:

Situation	Pages checked
No ToC found	Check up to <code>toc_check_page_num</code> pages, then return <code>toc_page_list = []</code> .
ToC found early	Keep scanning until the first non-ToC page after the ToC block.
ToC spans past page 20	Continue past page 20 while pages are still labeled as ToC.

3. **Detect page numbers in the ToC:**

- (a) Concatenate the detected ToC pages and normalize dot leaders. Then ask the LLM whether section page numbers exist.

Input: Concatenated ToC text.

```
Contents
1. Introduction ..... 3
2. Monetary Policy ..... 9
3. Financial Stability ..... 21
```

vs.

```
Contents
1. Introduction
2. Monetary Policy
3. Financial Stability
```

LLM Output: Whether page numbers are provided in the ToC.

```
page_index_given_in_toc: "yes"
```

vs.

```
page_index_given_in_toc: "no"
```

Remark:

- Dot leaders such as are normalized before the LLM call.
- If `page_index_given_in_toc: "yes"`, the existing ToC is used as the structure source.
- If `page_index_given_in_toc: "no"`, the implementation may check later ToC pages within `toc_check_page_num`; if no usable page-number ToC is found, the top-level path falls back to generating the tree from full page-marked text.

4. Construct PageIndex Tree:

- **ToC + page numbers found:** Normalize the ToC into JSON, map printed page numbers to physical PDF pages, compute the page offset, and fill missing physical indexes.

(a) Convert ToC to JSON.

Input: Concatenated detected ToC pages (plain ToC text).

LLM Output: A flat JSON list.

```
[
  {"structure": "1", "title": "Introduction", "page": 3},
  {"structure": "2", "title": "Monetary Policy", "page": 9},
  {"structure": "3", "title": "Financial Stability", "page": 21}
]
```

(b) Calibrate the difference between ToC page numbers and PDF page extracted positions.

Input: A copy of the normalized ToC JSON + at most `toc_check_page_num` pages after the ToC, wrapped with physical page tags. The implementation uses this to ask where early ToC titles actually appear in the PDF.

```
[
  {"structure": "1", "title": "Introduction", "page": 3},
  {"structure": "2", "title": "Monetary Policy", "page": 9},
  {"structure": "3", "title": "Financial Stability", "page": 21}
]

<physical_index_4>
Introduction ...
<physical_index_4>

<physical_index_5>
...
<physical_index_5>
```

LLM Output: Physical PDF pages that contain the start of matching ToC titles.

```
[
  {"structure": "1", "title": "Introduction", "physical_index": 8},
  {"structure": "2", "title": "Monetary Policy", "physical_index": 14}
]
```

- (c) Match entries by title and compute the offset with normal code (Rules based).

```
Introduction:      physical_index 8 - printed page 3 = offset 5
Monetary Policy:   physical_index 14 - printed page 9 = offset 5

physical_index = printed page + most common offset
```

- (d) Apply the offset to every ToC entry and fill missing physical indexes by asking the LLM to locate them within pages bounded by nearby known sections.
- (e) Verify and repair the flat section list before tree conversion. This is the step that turns a predicted flat list into a **verified** flat list.
- Remove or ignore **physical_index** values that are outside the real document page range.
 - For each predicted section item, load its predicted page text. Then ask the LLM whether the section title appears on that page.
 - Compute the verification accuracy from those title-page checks.
 - If accuracy is 1.0, accept the flat list.
 - If accuracy is above 0.6 but some sections are wrong, repair the incorrect page indexes up to 3 attempts.
 - If accuracy is too low in the ToC-with-page-numbers path, fall back to mapping the ToC titles from page-marked text; if that also fails, use the no-ToC generation path.
 - After the list is accepted, check whether each section title starts at the beginning of its page and store the result as **appear_start**. This is later used when computing **end_index**.

LLM check input: One predicted section item plus the extracted text from its predicted physical page.

```
section_item = {
  "structure": "2",
  "title": "Monetary Policy",
  "physical_index": 14
}

page_text loaded from physical_index 14:
<physical_index_14>
Monetary Policy ...
<physical_index_14>

-> answer: "yes"
```

Output: Verified flat section list.

```
[
  {"structure": "1", "title": "Introduction",
   "physical_index": 8, "appear_start": "yes"},
  {"structure": "2", "title": "Monetary Policy",
   "physical_index": 14, "appear_start": "yes"}
]
```

- (f) Convert the verified flat list into the PageIndex tree. This step is rules-based and happens after section starts have been produced, checked, and repaired when needed.
- Each **physical_index** becomes the node's **start_index**.

- Each **end_index** is derived from the next section's **physical_index**. If the next section title appears at the start of its page, the previous node ends on the page before it; otherwise it may share that page.
- The **structure** value controls nesting: 2.1 becomes a child of 2, and 2.1.1 becomes a child of 2.1.

Input: Verified flat section list.

```
[
  {"structure": "1", "title": "Introduction",
   "physical_index": 8, "appear_start": "yes"},
  {"structure": "2", "title": "Monetary Policy",
   "physical_index": 14, "appear_start": "yes"},
  {"structure": "2.1", "title": "March Summary",
   "physical_index": 16, "appear_start": "yes"},
  {"structure": "3", "title": "Financial Stability",
   "physical_index": 21, "appear_start": "yes"}
]
```

Output: Nested PageIndex tree with page ranges.

```
[
  {"title": "Introduction", "start_index": 8,
   "end_index": 13},
  {"title": "Monetary Policy", "start_index": 14,
   "end_index": 15, "nodes": [
    {"title": "March Summary", "start_index": 16,
     "end_index": 20}
  ]},
  {"title": "Financial Stability", "start_index": 21,
   "end_index": 31}
]
```

- **No usable ToC:** If no ToC is found, or if the detected ToC has no page numbers, generate the tree from page-marked document text.

- Wrap document pages with physical page tags.

```
<physical_index_1>
Annual Report 2023 ...
<physical_index_1>

<physical_index_2>
Financial Stability ...
<physical_index_2>
```

- Group the page-marked text by token budget so each LLM call stays within context limits. The grouping is based on token count, not a fixed page count.

Criteria:

- If the total document text is within **max_tokens** (20,000 by default), all pages stay in one group.
- If the document is longer, the system estimates how many groups are needed and computes a target token budget for each group.
- A new group starts when adding the next page would exceed that target budget.
- Neighboring groups keep a one-page overlap by default, so section boundaries near a group edge are less likely to be lost.

```
group_1: <physical_index_1> ... <physical_index_12>
group_2: <physical_index_12> ... <physical_index_24>
```

- Ask the LLM to generate section titles, hierarchy, and physical start pages from each group. Each LLM call receives one page-marked group.

Input: One group of page-marked document text.

```
<physical_index_1>
```

```
Annual Report 2023 ...
Introduction ...
<physical_index_1>
```

```
...
```

```
<physical_index_21>
Financial Stability ...
<physical_index_21>
```

LLM Output: A flat section list detected from that group.

```
[
  {"structure": "1", "title": "Introduction",
   "physical_index": "<physical_index_1>"},
  {"structure": "2", "title": "Financial Stability",
   "physical_index": "<physical_index_21>"}
]
```

- (d) Continue group by group and merge the partial structures. The first group creates the initial flat list; later groups extend that list.

```
group_1 output + group_2 output + ...
-> merged flat section list
```

- (e) Convert physical page tags into numeric indexes.

```
[
  {"structure": "1", "title": "Introduction",
   "physical_index": 1},
  {"structure": "2", "title": "Financial Stability",
   "physical_index": 21}
]
```

- (f) Verify and repair the merged flat list using the same hybrid verification logic: rule-based range checks, LLM title-page checks, accuracy calculation, and repair attempts. If this no-ToC path still has low accuracy, the implementation raises a processing failure rather than trying another indexing path.

```
section_item = {
  "structure": "2",
  "title": "Financial Stability",
  "physical_index": 21
}

page_text loaded from physical_index 21:
<physical_index_21>
Financial Stability ...
<physical_index_21>

-> answer: "yes"
```

- (g) Check whether each accepted section starts at the beginning of its page and store **appear_start**.

```
[
  {"structure": "1", "title": "Introduction",
   "physical_index": 1, "appear_start": "yes"},
  {"structure": "2", "title": "Financial Stability",
   "physical_index": 21, "appear_start": "yes"}
]
```

- (h) Convert the verified flat list into the nested PageIndex tree. This is rules-based: **physical_index** becomes **start_index**, **end_index** is derived from the next section start, and **structure** controls nesting.

```
[
  {"title": "Introduction", "start_index": 1,
   "end_index": 20},
  {"title": "Financial Stability", "start_index": 21,
```

```
"end_index": 31}
]
```

- **Fallback after ToC + page numbers fails verification:** If the ToC-with-page-numbers path fails verification, reuse the detected ToC titles and map them from token-bounded groups of page-marked document text. If that also fails verification, fall back to no-ToC generation.

- (a) Re-normalize the detected ToC text into JSON and ask the LLM to locate section starts from each group of page-marked document text. In this fallback, the physical positions come from document pages rather than from the printed-page offset.

```
[
  {"structure": "1", "title": "Introduction"},
  {"structure": "2", "title": "Monetary Policy"},
  {"structure": "3", "title": "Financial Stability"}
]
```

```
<physical_index_1>
...
<physical_index_1>

<physical_index_8>
Introduction ...
<physical_index_8>
```

- (b) The LLM returns physical start pages for the ToC titles.

```
[
  {"structure": "1", "title": "Introduction",
   "physical_index": 8},
  {"structure": "2", "title": "Monetary Policy",
   "physical_index": 14}
]
```

- (c) Verify the mapped physical indexes. If verification still fails, run the no-ToC generation path.
- (d) This fallback is used after the page-number path fails; it is not the normal top-level path for `page_index_given_in_toc`: "no".

5. **Post-process and split large nodes:** Convert the verified flat section list into the final nested PageIndex tree. Basic post-processing is rules-based; large-node splitting can trigger new LLM calls.

- (a) Convert the verified flat list into page ranges and nested **nodes**.

Input: Verified flat section list.

```
[
  {"structure": "1", "title": "Introduction",
   "physical_index": 1, "appear_start": "yes"},
  {"structure": "2", "title": "Financial Stability",
   "physical_index": 21, "appear_start": "yes"}
]
```

LLM Output: None. This is rules-based.

Code Output: Nested tree with **start_index**, **end_index**, and child **nodes**.

```
[
  {"title": "Introduction", "start_index": 1,
   "end_index": 20},
  {"title": "Financial Stability", "start_index": 21,
   "end_index": 31}
]
```

- (b) Decide whether each node is too large.

Input: One node's page span and token count from the extracted page list.

```
node = {
  "title": "Financial Stability",
  "start_index": 21,
  "end_index": 35
}
```

```
page_span = 14
token_count = 24000
```

LLM Output: None. This is rules-based.

Rule: Split only when both conditions are true.

```
page_span > max_page_num_each_node
and
token_count >= max_token_num_each_node
```

- (c) If a node is too large, re-index that node's page range using the no-ToC path.

Input: The large node's page text, wrapped with physical page tags and grouped by token budget.

```
<physical_index_21>
Financial Stability ...
<physical_index_21>

...

<physical_index_35>
Conclusion of this section ...
<physical_index_35>
```

LLM Output: A deeper flat section list for that node.

```
[
  {"structure": "1", "title": "Monitoring Financial Vulnerabilities",
   "physical_index": "<physical_index_22>"},
  {"structure": "2", "title": "Domestic and International Cooperation",
   "physical_index": "<physical_index_28>"}
]
```

Code Output: Verified child nodes are inserted under the large parent node.

```
{
  "title": "Financial Stability",
  "start_index": 21,
  "end_index": 22,
  "nodes": [
    {"title": "Monitoring Financial Vulnerabilities",
     "start_index": 22, "end_index": 27},
    {"title": "Domestic and International Cooperation",
     "start_index": 28, "end_index": 35}
  ]
}
```

6. **Attach retrieval fields:** Add optional fields used by retrieval and answering. Some fields are added by code; summaries and document description use LLM calls when enabled.

- (a) Add **node_id**.

Input: Nested PageIndex tree without ids.

```
[
  {"title": "Introduction", "start_index": 1,
   "end_index": 20}
]
```

LLM Output: None. This is rules-based.

Code Output: Tree with zero-padded node ids.

```
[
  {"title": "Introduction", "node_id": "0000",
   "start_index": 1, "end_index": 20}
]
```

```
]
```

- (b) Add raw node **text**.

Input: Node page range plus cached extracted page text.

```
node = {"start_index": 1, "end_index": 2}

page_list = [
    ("Annual Report 2023 ...", 742),
    ("Introduction ...", 615)
]
```

LLM Output: None. This is copied by code.

Code Output: The node receives concatenated raw text.

```
{
    "title": "Introduction",
    "text": "Annual Report 2023 ... Introduction ..."
}
```

- (c) Generate node **summary** when enabled.

Input: One node's raw **text**.

```
Partial Document Text:
Financial Stability ...
```

LLM Output: A concise summary stored on that node.

```
{
    "summary": "This section discusses financial stability,
    vulnerabilities, monitoring, and coordination."
}
```

Remark: If raw **text** is not requested in the final tree, the implementation can temporarily attach text for summary generation and remove it afterward.

- (d) Generate document description when enabled.

Input: A clean version of the tree containing fields such as **title**, **node_id**, **summary**, and child **nodes**.

```
[
    {
        "title": "Introduction", "node_id": "0000",
        "summary": "Overview of the report."
    },
    {
        "title": "Financial Stability", "node_id": "0006",
        "summary": "Discussion of financial stability."
    }
]
```

LLM Output: One sentence that describes the whole document.

```
doc_description =
"A Federal Reserve annual report covering policy,
financial stability, supervision, and institutional activities."
```

Remark: In the current PDF implementation, document description generation happens after node summaries when both are enabled.

- (e) Store the document record.

Input: Final tree, document path, document name, page count, cached page text, and optional document description.

```
{
    "path": "/abs/path/annual-report-2023.pdf",
    "doc_name": "annual-report-2023.pdf",
    "page_count": 31,
    "structure": [ ... ],
    "pages": [ ... ]
}
```

LLM Output: None. This is storage code.

Code Output: A retrieval-ready document record with a returned **id**.

```
{
    "id": "uuid-doc-id",
    ...
}
```

```

"type": "pdf",
"path": "/abs/path/annual-report-2023.pdf",
"doc_name": "annual-report-2023.pdf",
"doc_description": "A Federal Reserve annual report ...",
"page_count": 31,
"structure": [ ... ],
"pages": [ ... ]
}

```

- **Nodes** represent a section (e.g. chapter, paragraph, page)

```

Node {
  node_id: string,           // Optional unique node identifier
  title: string,             // Human-readable section title
  start_index: integer,      // Start physical PDF page
  end_index: integer,        // End physical PDF page
  summary: string,           // Optional LLM-generated node summary
  text: string,              // Optional raw section text
  nodes: [Node]              // Optional child nodes
}

```

- **node_id**: Serves as a reference key to locate the corresponding raw data.
- **nodes**: Field allows recursive nesting, forming a full PageIndex tree.
- **text** and **summary**: Optional fields controlled by indexing settings.

- **ToC Example:**

```

...
{
  "node_id": "0006",
  "title": "Financial Stability",
  "start_index": 21,
  "end_index": 22,
  "summary": "The Federal Reserve ...",
  "nodes": [
    {
      "node_id": "0007",
      "title": "Monitoring Financial Vulnerabilities",
      "start_index": 22,
      "end_index": 28,
      "summary": "The Federal Reserve's monitoring ..."
    },
    {
      "node_id": "0008",
      "title": "Domestic and International Cooperation and Coordination",
      "start_index": 28,
      "end_index": 31,
      "summary": "In 2023, the Federal Reserve collaborated ..."
    }
  ]
}
...

```

- **start_index / end_index**: Start page / End page

- Each node in the PageIndex tree links to corresponding **raw content** (e.g. text, images, tables)

`node_id` → node object → `start_index` / `end_index` → cached page `text`

- A node-id mapping can be created from the tree for quick lookup.
- Query-time tools usually give the LLM the compact tree without raw `text`, then retrieve cached page text using the selected page range.

Indexing Step 2: Create Document Record (LLM Optional)

The local client creates a document record after indexing. The record step is mostly storage; it may include LLM-generated node summaries and a document-level description that were produced during indexing.

- **Main fields:** `id`, `type`, `path`, `doc_name`, `doc_description`, page or line count, `structure`, and cached page text for PDFs.
- **Description:** A one-sentence document-level summary generated from a clean version of the structure when `if_add_doc_description` is enabled.
 - In the provided client, PDF and Markdown indexing enable document description generation.
 - The default lower-level config can leave document description disabled unless the caller enables it.
- **Workspace storage:** If a workspace is configured, the document JSON is written to `<doc_id>.json`; a smaller `_meta.json` stores lightweight metadata for startup.

Implementation detail:

1. **Create a document id:** Each indexing call creates a new UUID. If a workspace is used, that id is persisted and can be reloaded later.
2. **Store core metadata:** The record stores the absolute source `path`, document `type`, `doc_name`, optional `doc_description`, and page or line count.
3. **Store the structure:** The full record contains the PageIndex `structure`. When saved to a workspace for PDFs, raw node `text` is stripped from the structure because page text is cached separately.
4. **Cache raw page evidence:** For PDFs, the client stores `pages`, a list of page numbers and extracted text. Query-time retrieval can return selected page ranges without reopening the PDF.
5. **Save lightweight metadata:** Workspace mode also writes `_meta.json`, which keeps only metadata such as type, name, description, source path, and page or line count.

Document-Level Record Example:

```
{
  "id": "uuid-doc-id",
  "type": "pdf",
  "path": "/abs/path/annual-report-2023.pdf",
  "doc_name": "annual-report-2023.pdf",
  "doc_description": "One-sentence LLM description.",
  "page_count": 31,
  "structure": [ ... ],
```

```
"pages": [  
  {"page": 1, "content": "Annual Report 2023 ..."}  
]
```

`doc_id` → document record → structure / cached pages / source file path

Indexing Step 3: Build Optional Document-Search Index

This step is useful when the system has many documents, but it is not a built-in SQL/vector index in the local `pageindex` package. The package provides document records and retrieval helpers; a production system can build an external document-search index from those records.

- **SQL metadata search:** Good for exact filters such as year, source, company, document type, or permission when those fields exist in the external index.
- **Vector search:** Good for semantic matching against document descriptions, titles, abstracts, or top-level summaries.

Implementation detail:

1. **Choose searchable fields:** An external index can use `doc_name`, `doc_description`, `type`, page or line count, and optionally high-level tree summaries.
2. **Build metadata filters:** Store searchable document fields in an external database for exact filters.
3. **Build semantic search:** Document descriptions or high-level summaries can be embedded into vectors outside the core package.
4. **Combine retrieval signals:** Production systems often combine metadata filters, keyword search, vector similarity, and optional LLM reranking before tree search.
5. **Return candidate documents:** The output is a small ranked set of `doc_id` values. The local retrieval helpers expect a known `doc_id`.
6. **Handle scale:** For small collections, this step can be simple or skipped. For large collections, it avoids running tree search over every document.
7. **Keep the index synchronized:** When documents are added, deleted, or re-indexed, the external document-search index should update together with the `PageIndex` workspace records.

Query Step 1: Document Search (External or Agent-Level)

For multiple documents, an external index or an agent chooses candidate `doc_id` values. The local package itself exposes document metadata by id, but does not rank all documents by query.

- **Input:** User question, optional filters, and available document metadata.
- **Output:** A ranked list of candidate `doc_id` values.

Query Step 2: Tree Search (Can Use Same or Retrieval LLM)

The retrieval model or agent reads the PageIndex tree for each selected document, usually without raw node text, and selects relevant nodes or page ranges.

- **Input:** User question, selected `doc_id`, and that document's compact tree with raw text removed.
- **Output:** Relevant `node_id` values or page ranges, often with short reasons.

Query Step 3: Retrieve Raw Content

After tree search selects nodes or page ranges, the system retrieves the corresponding raw evidence.

In the local retrieval helpers, raw evidence is fetched by page range from cached pages. A node-id map can translate selected `node_id` values into page ranges.

Query Step 4: Answer the Question (Can Use Same or Answering LLM)

- **Input:** User question, retrieved evidence, page numbers, `doc_id`, and optional `node_id`.
- **Output:** A grounded answer with citations to pages, nodes, or documents.

Different Methods

Document Search vs. Tree Search

In a multi-document system, retrieval is usually split into two levels.

- **Document search:** Find the relevant document records. The output is one or more `doc_id` values.
- **Tree search:** Search inside each selected document's PageIndex tree. The output is one or more `node_id` values or page ranges.

```
query → doc-search → doc_id → tree-search → node_id → raw content → answer
```

- **Document search tutorial** (`doc-search`): Shows how to choose documents using meta-data, semantics, or LLM-generated descriptions.
- **Tree search tutorial** (`tree-search`): Shows how to choose relevant nodes inside one known document.

Manual Text RAG: `demo/PageIndex_Simple`

This demo starts at the **tree-search** level because the document is already known.

1. **Convert PDF to PageIndex tree:** PageIndex creates node titles, summaries, page indexes, raw text, and node ids.
2. **Tree search:** Give the LLM the question and the tree without raw text. The LLM returns relevant `node_id` values.
3. **Map nodes to content:** Use a node-id mapping or the node's `start_index`/`end_index` range to locate the full content.
4. **Generate answer:** Feed only the selected node text to the LLM to answer.

Production note: For many documents, add a document-search step before this flow.

Vision RAG: [demo/Vision_PageIndex](#)

This demo also starts from one known document, but the final answer is generated from rendered page images.

1. **Convert PDF to PageIndex tree:** PageIndex creates node titles, summaries, page indexes, raw text, and node ids.
2. **Render pages:** Convert PDF pages into image files.
3. **Tree search:** Use a multimodal model as a text-only model to select nodes from the tree.
4. **Collect page images:** Convert selected nodes into page ranges and get page images.
5. **Generate visual answer:** Send the question and selected page images to a vision-language model.

Production note: Use this when figures, tables, scanned pages, or visual layout are important.

Chat Quickstart: [demo/Chat_Quickstart_PageIndex](#)

This demo hides the tree-search and answer-generation details behind the PageIndex chat API.

1. **Submit document:** Upload a PDF and keep the returned `doc_id`.
2. **Check status:** Wait until PageIndex finishes processing.
3. **Ask question:** Call `pi_client.chat_completions(...)` with the user question and `doc_id`.
4. **Stream answer:** PageIndex handles retrieval and answer generation internally.

Agentic Retrieval: [demo/Agentic_PageIndex](#)

Agentic retrieval is not just “LLM retrieves then LLM answers.” Normal PageIndex can also do that. The difference is that retrieval becomes a **tool** inside a larger agent workflow.

1. **Submit document:** Upload a PDF and keep `doc_id`.
2. **Ask retrieval prompt:** Request raw relevant evidence in a structured JSON format.
3. **Parse JSON:** Extract page numbers and evidence content from the model output.
4. **Check / reuse evidence:** An agent can validate, compare, call more tools, retrieve again, or pass evidence to another step.
5. **Generate final response if needed:** The final answer may be produced after the evidence is collected.

Key distinction:

- **Normal PageIndex:** Retrieval is a step inside a fixed QA pipeline.
- **Agentic PageIndex:** Retrieval is an iterative tool that an agent can call conditionally.

Comparison

Method	Main Role	Flow Summary
Document Search	Select document(s)	Query → document index → doc_id
Tree Search	Select section(s)	doc_id → PageIndex tree → node_id
Manual Text RAG	Custom text QA	Tree → node ids → node text → answer
Vision RAG	Visual document QA	Tree → node ids → page images → VLM answer
Chat Quickstart	Simple hosted QA	Document id → PageIndex chat API → streamed answer
Agentic Retrieval	Structured evidence tool	Query → retrieval prompt → JSON evidence → agent use